Introducción a la Programación con Java y Eclipse

Unidad 1. Prueba de aplicaciones web y para dispositivos móviles

José Gaspar Sánchez García

<2026-02-19 jue>

Contents

1 Descarga e instalación del Entorno de Desarrollo Integrado Eclipse **2**

2 ¿Qué es la programación estructurada? **2**

3 Java como lenguaje estructurado **4**

4 Estructura básica de un programa en Java **4**

5 Variables y tipos de datos **5**
5.1 Tabla de tamaños de tipos primitivos 6

6 Entrada y salida básica **6**

7 Estructuras de control **6**
7.1 Secuencia 6
7.2 Selección 7
7.2.1 if / else 7
7.2.2 switch 7
7.3 Iteración 7
7.3.1 for 7
7.3.2 while 7
7.3.3 do-while 7

8 Funciones (métodos) **8**

9 Buenas prácticas en programación estructurada **8**

10 Ejemplo completo **8**

11 Ejercicios propuestos **9**

12 Entrega: **12**

13 Enlaces de interés **12**

1 Descarga e instalación del Entorno de Desarrollo Integrado Eclipse

1. Descargamos el IDE Eclipse desde su página web oficial (`https://www.eclipse.org/downloads/`).

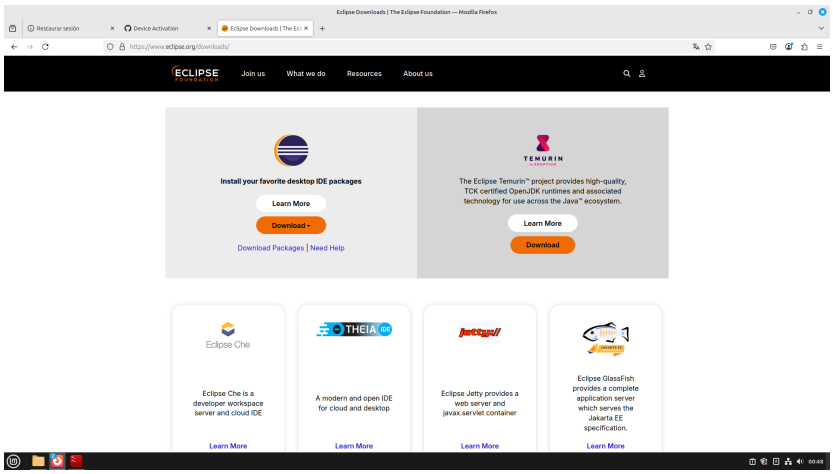


Figure 1: Página oficial de Eclipse.

2. Descomprimimos el archivo descargado.

```
tar xzvf *.gz
cd eclipse-installer
./eclipse-inst
```

3. Instalamos Eclipe IDE for Java Developers.

4. Creamos un nuevo proyecto Maven en Eclipse llamado *IntroJavaEclipse-INICIALES*.

2 ¿Qué es la programación estructurada?

La programación estructurada es un paradigma que organiza el código en bloques lógicos para facilitar la lectura, el mantenimiento y la detección de errores.

Se basa en tres estructuras fundamentales:

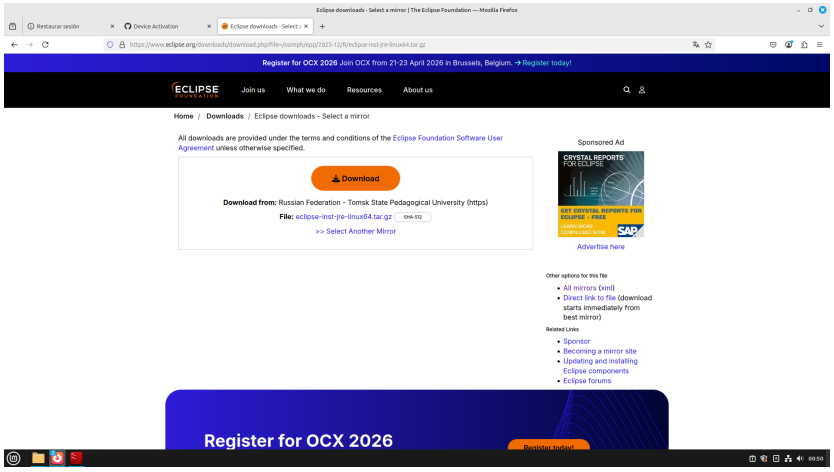


Figure 2: Descarga de Eclipse.

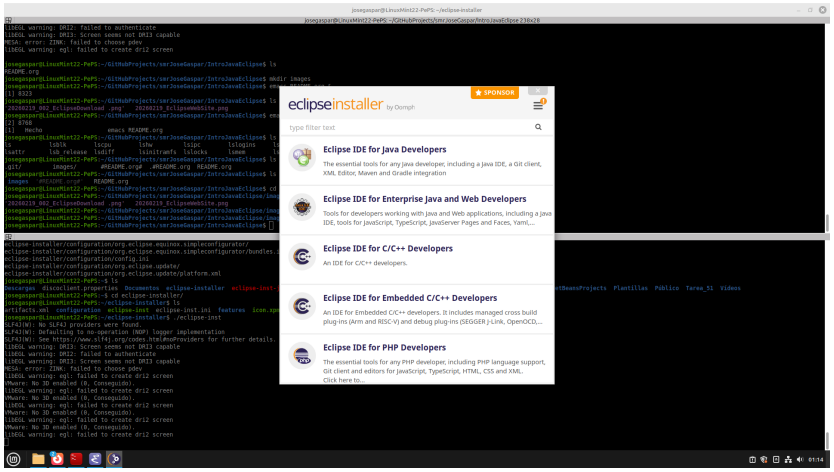


Figure 3: Instalador de Eclipse.

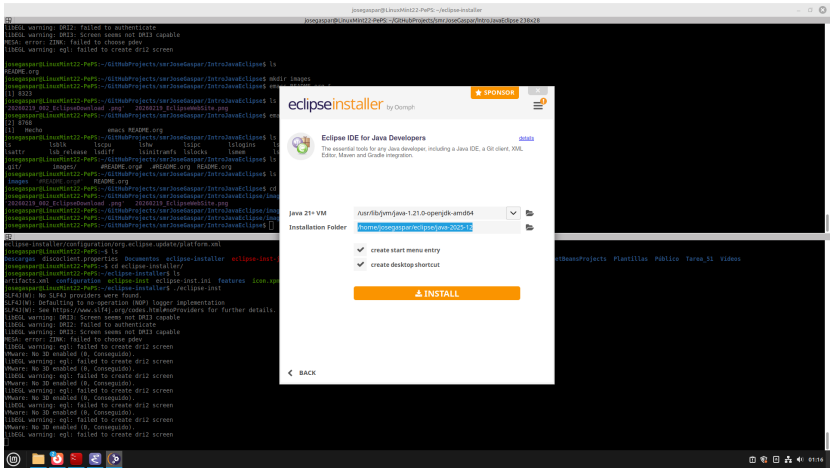


Figure 4: Pantalla del instalador de Eclipse.

- **Secuencia**: instrucciones que se ejecutan en orden.
- **Selección**: decisiones (if, switch).
- **Iteración**: repeticiones (for, while, do-while).

Su objetivo es evitar el uso excesivo de saltos como `goto` y promover un código más claro y modular.

3 Java como lenguaje estructurado

Aunque Java es un lenguaje orientado a objetos, permite trabajar perfectamente con programación estructurada, especialmente al inicio del aprendizaje.

Características relevantes:

- Sintaxis clara y similar a C/C++.
- Tipado fuerte.
- Uso obligatorio de clases, pero el flujo del programa puede ser estructurado.
- Uso de bloques `{ }` para delimitar estructuras.

4 Estructura básica de un programa en Java

```
public class Main {
    public static void main(String[] args) {
        // Código aquí
```

```
    }
}
```

Figure 5: Estructura básica de un programa en java

Elementos clave:

- **class**: todo programa debe estar dentro de una clase.
- **main**: punto de entrada del programa.
- **{ }**: delimitan bloques de código.

5 Variables y tipos de datos

Este ejemplo muestra distintos tipos de variables en Java y el tamaño que ocupa cada tipo primitivo según la especificación del lenguaje.

```
public class Ejemplo002 {

      public static void main(String[] args) {
      // TODO Auto-generated method stub

      // Tipos primitivos numéricos enteros
          byte miByte = 10;          // 1 byte  (8 bits)
          short miShort = 200;       // 2 bytes (16 bits)
          int miInt = 50000;         // 4 bytes (32 bits)
          long miLong = 100000L;     // 8 bytes (64 bits)

          // Tipos primitivos numéricos decimales
          float miFloat = 5.75f;     // 4 bytes (32 bits)
          double miDouble = 19.99;   // 8 bytes (64 bits)

          // Tipo primitivo carácter
          char miChar = 'A';         // 2 bytes (16 bits, Unicode)

          // Tipo primitivo booleano
          boolean miBoolean = true;  // No estrictamente definido

          // Tipo de referencia
          String miString = "Hola Java"; // Tamaño variable

          // Mostrar valores
          System.out.println("byte: " + miByte+". Tamaño: "+ Byte.SIZE+" bytes. Mínimo: "+
          System.out.println("short: " + miShort+". Tamaño: "+Short.SIZE+" bytes. Mínimo:
          System.out.println("int: " + miInt+". Tamaño: "+Integer.SIZE+" bytes. Mínimo: "+
          System.out.println("long: " + miLong+". Tamaño: "+Long.SIZE+" bytes. Mínimo: "+I
```

```
            System.out.println("float: " + miFloat+". Tamaño: "+Float.SIZE+ "bytes. Mínimo:
            System.out.println("double: " + miDouble+". Tamaño: "+Double.SIZE+ "bytes. "+Dou
            System.out.println("char: " + miChar+". Tamaño: "+Character.SIZE+" bytes. Mínimo
            System.out.println("boolean: " + miBoolean+". Tamaño: 1 bit");
            System.out.println("String: " + miString+". Tamaño: ?");
        }
}
```

Figure 6: Variable y tipos de datos

5.1 Tabla de tamaños de tipos primitivos

Tipo	Tamaño	Rango aproximado
byte	1 byte	-128 a 127
short	2 bytes	-32,768 a 32,767
int	4 bytes	-2,147,483,648 a 2,147,483,647
long	8 bytes	$\pm 9 \times 10^{18}$
float	4 bytes	~6–7 dígitos decimales
double	8 bytes	~15 dígitos decimales
char	2 bytes	Caracter Unicode
boolean	N/D	true / false
String	Variable	Depende del contenido

6 Entrada y salida básica

Salida por consola:

```
System.out.println("Hola mundo");
```

Entrada con Scanner:

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
int numero = sc.nextInt();
String texto = sc.nextLine();
```

7 Estructuras de control

7.1 Secuencia

```
public class Ejemplo003 {
        /**
         * @param args
         */
```

```
        public static void main(String[] args) {
                // TODO Auto-generated method stub
                int a = 5;
                int b = 3;
                int suma = a + b;
                System.out.println("a = "+a+", b = "+b+ ", suma = "+suma);
        }
}
```

7.2 Selección

7.2.1 if / else

```
if (edad >= 18) {
    System.out.println("Mayor de edad");
} else {
    System.out.println("Menor de edad");
}
```

7.2.2 switch

```
switch (dia) {
    case 1: System.out.println("Lunes"); break;
    case 2: System.out.println("Martes"); break;
    default: System.out.println("Otro día");
}
```

7.3 Iteración

7.3.1 for

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

7.3.2 while

```
while (contador < 10) {
    contador++;
}
```

7.3.3 do-while

```
do {
    opcion = sc.nextInt();
} while (opcion != 0);
```

8 Funciones (métodos)

```
public static int sumar(int a, int b) {
    return a + b;
}
```

Llamada:

```
int resultado = sumar(3, 4);
```

Ventajas:

- Reutilización.
- Claridad.
- Mantenimiento más sencillo.

9 Buenas prácticas en programación estructurada

- Usar nombres descriptivos.
- Evitar bloques demasiado largos.
- Comentar solo lo necesario.
- Mantener una indentación consistente.
- Dividir el código en métodos pequeños.

10 Ejemplo completo

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce un número: ");
        int n = sc.nextInt();

        if (n % 2 == 0) {
            System.out.println("Es par");
        } else {
            System.out.println("Es impar");
        }

        System.out.println("Contando hasta " + n);
```

```
        for (int i = 1; i <= n; i++) {
            System.out.println(i);
        }
    }
}
```

11 Ejercicios propuestos

Ejercicios de Secuencia

1. Escribe un programa que pida dos números y muestre:
   - Su suma
   - Su resta
   - Su multiplicación
   - Su división
2. Pide al usuario su nombre y edad y muestra un mensaje como: *"Hola Ana, tienes 20 años."*
3. Convierte grados Celsius a Fahrenheit usando la fórmula: $F = C \times 1.8 + 32$
4. Calcula el área de un triángulo pidiendo base y altura.

   Ejercicios de Selección (if / else / switch)
5. Pide un número e indica si es positivo, negativo o cero.
6. Pide una nota (0–10) e imprime:
   - Insuficiente
   - Suficiente
   - Bien
   - Notable
   - Sobresaliente
7. Pide un número del 1 al 7 y muestra el día de la semana usando `switch`.
8. Pide un año y determina si es bisiesto.
9. Pide tres números y muestra cuál es el mayor.

   Ejercicios de Bucles (for, while, do-while)
10. Muestra los números del 1 al 100 usando:
    - un `for`
    - un `while`

- un `do-while`

11. Pide un número y muestra su tabla de multiplicar del 1 al 10.
12. Pide números al usuario hasta que introduzca un 0. Al final, muestra la suma total.
13. Muestra los primeros 20 números pares.
14. Calcula el factorial de un número usando un `for`.
15. Pide un número N y muestra todos los números del 1 al N que sean múltiplos de 3.

Ejercicios con Métodos (Funciones)

16. Crea un método `esPar(int n)` que devuelva `true` si el número es par.
17. Crea un método `maximo(int a, int b)` que devuelva el mayor de los dos.
18. Crea un método `saludar(String nombre)` que imprima un saludo personalizado.
19. Crea un método `potencia(int base, int exponente)` que calcule la potencia sin usar `Math.pow`.
20. Crea un método `esPrimo(int n)` que determine si un número es primo.
21. Crea un método `contarVocales(String texto)` que devuelva cuántas vocales tiene una cadena.

Ejercicios Integradores

22. **Menú interactivo** Crea un programa con un menú:
    - 1. Sumar dos números
    - 2. Restar
    - 3. Multiplicar
    - 4. Dividir
    - 0. Salir

    Usa `switch` y repite el menú hasta que el usuario elija salir.
23. **Contador de vocales:** Pide una frase y cuenta cuántas vocales tiene.
24. **Calculadora de notas:** Pide 5 notas y calcula:
    - La media
    - La nota más alta
    - La nota más baja

25. **Números primos hasta N:** Pide un número N y muestra todos los primos desde 1 hasta N usando un método `esPrimo`.

26. **Gestor simple de usuarios:** Pide nombres hasta que el usuario escriba "fin". Luego muestra:

    - Cantidad de nombres introducidos.
    - El nombre más largo.
    - El nombre más corto.

27. Implementa un programa que convierta números decimales a binarios sin usar funciones de Java.

28. Simula un cajero automático con opciones:

    - Consultar saldo.
    - Ingresar dinero.
    - Retirar dinero.
    - Salir.

29. Genera una secuencia Fibonacci de N términos.

    Ejercicios de Arrays en Java:

30. Leer 5 números por teclado y guardarlos en un array. Luego mostrar el mayor.

31. Sumar todos los elementos de un array de enteros.

32. Contar cuántos números pares hay en un array.

33. Invertir un array (sin usar estructuras auxiliares si quieres un reto).

34. Buscar un número dentro de un array y decir si existe.

35. Calcular la media de los valores de un array de tipo **entero**.

36. Comprobar si un array es palíndromo.

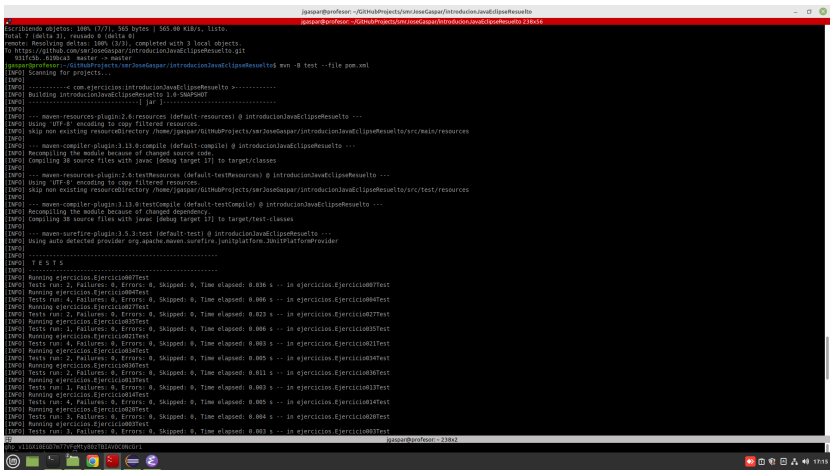
37. Contar repeticiones de cada número en un array.

38. **Adivina el número:** El programa genera un número aleatorio del 1 al 50. El usuario debe adivinarlo indicando si el número introducido es mayor o menor.

12 Entrega:

Debes entregar un documento PDF que contenga:

- Breve explicación de cómo has resuelto los ejercicios.
- Dirección URL del repositorio remoto GitHub donde estés ejecutando tu código.
- Captura de pantalla que muestre que los tests han sido ejecutados correctamente en local utilizando el comando `mvn -B test --file pom.xml`


jgaspar@profesor: ~/GitHubProjects/smrJoseGaspar/introducionJavaEclipseResuelto
jgaspar@profesor: ~/GitHubProjects/smrJoseGaspar/introducionJavaEclipseResuelto 238×16
Escribiendo objetos: 100% (7/7), 365 bytes | 565.00 KiB/s, listo.
Total 7 (delta 3), reusado 0 (delta 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/smrJoseGaspar/introducionJavaEclipseResuelto.git
931fc5b..619bca3 master -> master
jgaspar@profesor:~/GitHubProjects/smrJoseGaspar/introducionJavaEclipseResuelto$ mvn -B test --file pom.xml
[INFO] Scanning for projects...
[INFO]
[INFO] ----------< com.ejercicios:introducionJavaEclipseResuelto >----------
[INFO] Building introducionJavaEclipseResuelto 1.0-SNAPSHOT
[INFO] --------------------------------[jar]---------------------------------
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ introducionJavaEclipseResuelto ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /home/jgaspar/GitHubProjects/smrJoseGaspar/introducionJavaEclipseResuelto/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ introducionJavaEclipseResuelto ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 38 source files with javac [debug target 17] to target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ introducionJavaEclipseResuelto ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /home/jgaspar/GitHubProjects/smrJoseGaspar/introducionJavaEclipseResuelto/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:testCompile (default-testCompile) @ introducionJavaEclipseResuelto ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 38 source files with javac [debug target 17] to target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:3.5.3:test (default-test) @ introducionJavaEclipseResuelto ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JunitPlatformProvider
[INFO]
[INFO] ---
[INFO] T E S T S
[INFO] ---
[INFO] Running ejercicios.Ejercicio007Test
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in ejercicios.Ejercicio007Test
[INFO] Running ejercicios.Ejercicio004Test
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in ejercicios.Ejercicio004Test
[INFO] Running ejercicios.Ejercicio027Test
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.023 s -- in ejercicios.Ejercicio027Test
[INFO] Running ejercicios.Ejercicio035Test
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in ejercicios.Ejercicio035Test
[INFO] Running ejercicios.Ejercicio021Test
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s -- in ejercicios.Ejercicio021Test
[INFO] Running ejercicios.Ejercicio034Test
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in ejercicios.Ejercicio034Test
[INFO] Running ejercicios.Ejercicio036Test
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in ejercicios.Ejercicio036Test
[INFO] Running ejercicios.Ejercicio013Test
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s -- in ejercicios.Ejercicio013Test
[INFO] Running ejercicios.Ejercicio014Test
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in ejercicios.Ejercicio014Test
[INFO] Running ejercicios.Ejercicio020Test
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in ejercicios.Ejercicio020Test
[INFO] Running ejercicios.Ejercicio003Test
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s -- in ejercicios.Ejercicio003Test
jgaspar@profesor: ~ 238×2
PS1=$(dirtrim 2 | tr -d\t;date "+%H:%M:%S:%N:%Y:%m:%d:%A:%Z:%W:%Ds:%Dj:%Dd:%DW:%Dm:%Dy:%Dh:%Dm:%Ds:%Dn:%Dy:%Dh:%Dm:%Ds:%Dn:%Dy:%Dh:%Dm:%Ds:%Dn

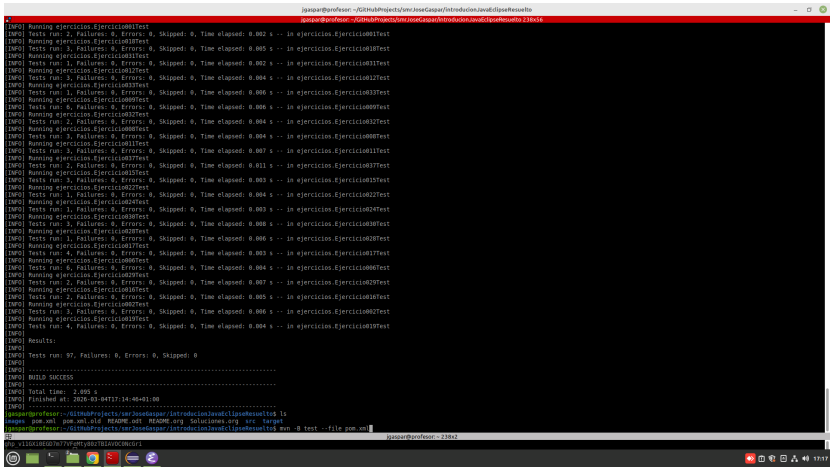


Figure 8: Captura de Pantalla de la Ejecución de los Tests en local.

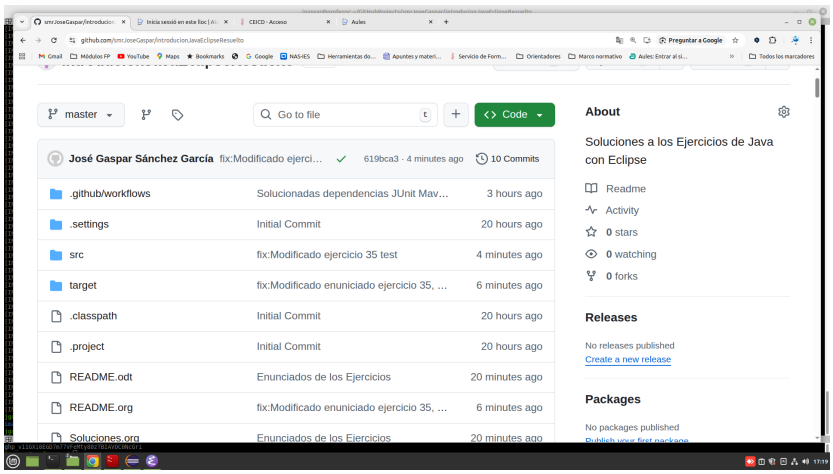


Figure 9: Captura de Pantalla de la Ejecución de los Tests en GitHub.

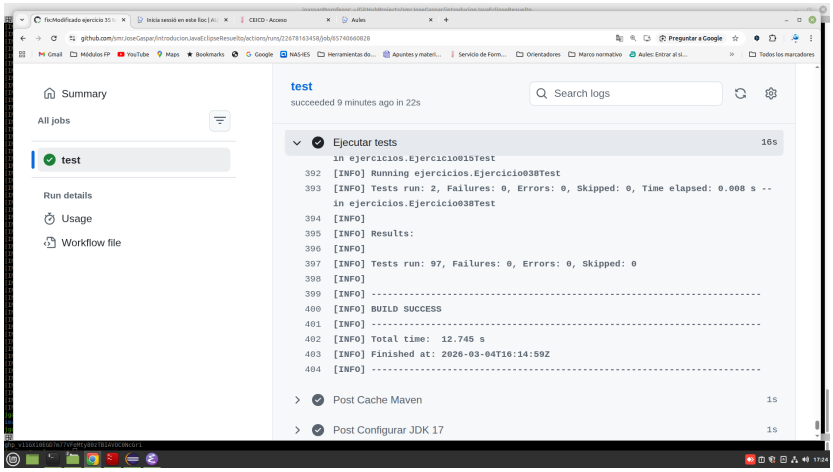


Figure 10: Captura de Pantalla de la Ejecución de los Tests en GitHub.